Assignment 2 — Student B: Q-Learning and stochastic experiments

Course: COSC/CPSC-4117EL-01 Artificial Intelligence, Fall 2026
Scope: Part 2 and the Q-Learning portion of Part 3 (group comparison with Value Iteration must be completed together).
Generative AI disclosure: ChatGPT (GPT-6) assisted with drafting and debugging the Python code, analyzing the experiment output, and drafting this report on November 4, 2026. **Before submission, insert the actual shared-conversation/prompt URL here and in all code comments:** [ADD REAL SHARE LINK]. Verify findings and comply with the course's separate D2L programming standards.

1. Environment and state representation

The program uses the instructor-provided `WarehouseGridWorld` class from `warehouse_gridworld_domain_random.py`, not an alternative grid simulator. The grid remains 10 × 10 with shelves, congestion cells, a pickup station, packing station, and charging dock. The framework suggests a state `(row, col, has_package, delivered)`. The last two bits distinguish the different task stages: `(0,0)` before pickup, `(1,0)` carrying a package, and `(1,1)` after delivery. This state captures both location and task progress; coordinates alone are insufficient for the Markov property because pickup, delivery, and dock rewards depend on previous task progress.

The environment is held at `layout_seed=42` so that all model variants use the same pickup station `(2,2)`, packing station `(0,4)`, and dock `(6,0)`. These are `(row, col)` coordinates, not `(x,y)`. The provided framework has deterministic `step(action)`. The adapter makes motion stochastic by sampling the executed action: intended action with probability `p`, and each perpendicular direction with probability `(1−p)/2`. It calls the original `step` with the sampled action. It does **not** reimplement rewards or obstacles. It keeps the instructor's per-step reward −1, +25 pickup, +40 delivery, +100 successful return, −10 entering congestion, and −3 invalid-move penalty (the step penalty also applies). On an invalid movement, the original framework leaves the robot in place. Episode success is recorded **only** for a `return_to_dock` event after delivery. The step cap is 120.

2. Q-Learning algorithm

A Q-table is initialized to zero for all encountered `(state,action)` pairs; unseen states also return a zero vector. The action space is `up`, `down`, `left`, `right` including commands that may hit obstacles. At each training step, the agent uses epsilon-greedy exploration and updates the selected state-action estimate:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

The future-value term is set to zero at terminal states and step-limit truncation. Epsilon starts at 1.0, decays multiplicatively every episode, and has a floor of 0.05. The Q-Learning agent does not receive any transition probabilities; only the environment-side adapter knows `p`. During evaluation, epsilon is 0, and the Q-table is frozen.

3. Experimental design

Two configurations were compared:

Configuration	Learning rate α	Discount γ	Epsilon decay	Minimum epsilon
A (fast decay)	0.20	0.95	0.995	0.05
B (slow decay)	0.10	0.99	0.999	0.05

For each `p ∈ {1.00, 0.90, 0.80}`, each configuration was trained for 5,000 episodes with each training seed in `{11, 42, 99}`: 18 separately trained agents. Each agent was evaluated on 100 independent motion-seeded episodes with greedy actions and no learning: 1,800 evaluation episodes total. Training and evaluation motion seeds do not regenerate the layout. Rewards, completion indicators, epsilon levels, greedy trajectories, policies, and training-state visitation counts are saved as files. The two configurations vary more than one hyperparameter, so comparisons are descriptive rather than isolating a single causal effect.

4. Results

The following numbers are **measured** from the included `evaluation_episodes.csv`. Each row aggregates 300 evaluation episodes: 100 evaluations for each of three training seeds. Success means the robot picked up, delivered, and returned to its dock within 120 steps.

Config	Intended action probability	Mean cumulative reward	Std. dev. reward	Success rate	Mean steps
A	1.00	143.00	0.00	100%	22.00
A	0.90	132.61	7.67	100%	25.11
A	0.80	119.36	14.06	100%	29.12
B	1.00	143.00	0.00	100%	22.00
B	0.90	132.61	7.67	100%	25.11
B	0.80	119.71	13.66	100%	29.02

At `p=1.00`, the 22-step trajectory obtains `25+40+100−22=143` because the successful path avoids congestion and invalid moves. As uncertainty increases, a chosen action is less likely to be executed and a robot may require extra attempts or detours. This increases step penalties and the chance of extra congestion penalties, lowering average return while success remains 100% on this fixed layout and test budget. These observations do **not** imply that all layouts or training runs will have 100% success.

Learning behaviour and robustness

The 500-episode mean rewards at the **start** versus the **end** of training (pooled across three seeds) were:

Config	p	Episodes 1–500	Episodes 4,501–5,000	Final 500 success
A	1.00	48.6	140.0	100%
A	0.90	22.2	128.7	100%
A	0.80	−2.6	113.5	100%

Config	p	Episodes 1–500	Episodes 4,501–5,000	Final 500 success
B	1.00	−170.6	139.9	100%
B	0.90	−205.0	129.1	100%
B	0.80	−245.0	113.8	100%

Both configurations improve substantially, with a broad plateau toward the end of training. Configuration B explores longer, producing lower early training rewards, yet its final greedy evaluations closely match A. Reward curves and moving success fractions are shown in `training_reward.png` and `training_success.png`. All three seeds reach 100% greedy evaluation success on the fixed map; this is a result of the tested seeds and settings rather than a guarantee of theoretical convergence. The success rate on the early training episodes is lower because epsilon-greedy exploration remains active.

Policy changes at lower p

`policy_grid.png` shows the greedy policy by task stage for configuration A and training seed 11 at `p=1.00` and `p=0.80`. The primary successful route is largely unchanged across these probabilities, but several peripheral states have different preferred actions. For example, at state `(8,7,1,1)` — on a congestion cell after delivery — A selects `right` at `p=1.00` but `down` at `p=0.80`. This same change occurs in all three training seeds; the state was encountered during training in both settings (visits by seed 11/42/99 at `p=1.00`: 16/15/14; at `p=0.80`: 17/6/17). Because this is a sparsely visited, off-route state, the direction change is an **observed learned-policy difference**, not proof that stochastic motion necessarily makes `down` optimal. More training, visits, and a Value Iteration comparison would be needed to establish the optimal action. `sample_routes.png` shows one actual stochastic evaluation trajectory for each probability, not a claim that all trajectories follow those exact paths.

5. Comparison with Value Iteration — group integration required

Value Iteration has access to the transition probabilities and reward model and can use the Bellman optimality equation to compute a policy. Q-Learning learns the action-value estimates from sampled interaction without receiving those probabilities. To complete the group's required head-to-head comparison, Student A should evaluate Value Iteration on **the same layout (seed 42), probabilities (1.00, 0.90, 0.80), maximum 120 steps, and the same episode-level evaluation seeds**. Compare reward, success fraction, path length, convergence pattern and task-stage policy arrows. The Q-Learning numbers above should not be represented as Value Iteration results.

6. Limitations

Experiments cover only one seeded warehouse layout, three training seeds, two jointly varied hyperparameter settings, and 100 evaluations per trained agent. A trained Q table can encode unstable actions for rarely visited states. The Markov representation excludes the remaining time budget; the step cap is handled as episode truncation, so behavior close to the time limit may differ from a finite-horizon time-augmented optimal policy. A larger sample of layouts and more isolated parameter sweeps would provide stronger generalization evidence. The separately provided D2L programming standards were not supplied for verification.

7. Student B contribution statement

Student B: integration of the provided warehouse framework with the required stochastic motion behaviour, Q-Learning algorithm, epsilon-greedy training, hyperparameter and random-seed experiments, greedy evaluation, policy/learning visualizations, Q-Learning result analysis and write-up. Student A is responsible for the MDP formulation, Value Iteration, and its experimental results. Joint contributions: algorithm comparison and final report/ZIP integration. Adjust this statement to accurately reflect the group's actual work before submission.